Software CaST

A (somewhat) minimalist Software for the solution of 2D-Elastic FEM problems

Author: Vítor Luís Costa Azevedo

Course: Introdução ao Método dos Elementos Finitos

Professor: DURAND, Raul

Date: 2026-07-08

Contents

1 Introduction

The development of **CaST** started as a project for the subject "Introdução ao Método dos Elementos Finitos" (*Introduction to the Finite Element Method*) at the university of Brasília.

As an open source software, it aims to present a minimalist UI for defining and calculating 2D solid and elastic elements, specifically **Constant Strain Triangles (CSTs)** which name the app. Its UI is defined in pure python, through the MVC (Model-View-Controller) architecture, purely based on the library *Tkinter*, and presents a certain easiness to be expanded into more complex calculations.

Currently, the Software supports the drawing of 2D elements via lines, circles and isoparametric 2nd degree parabolas. Supports and forces may be applied in geometric nodes and edges, in which forces, when applied to edges, can be defined as "normal" (pressure like) or "global", constant or trapezoidal.

The software also supports the saving and loading of designs, and visualization, through simple heatmaps, of stress, forces and displacements on elements.

2 Architecture

The MVC architecture, as described, was used in the making of the program. It was chosen, specially, because its set of rules define a workflow that avoids the creation of *Spaghetti* code, and facilitates the adding of new features.

In the MVC architecture, each section of code has a specific purpose, and those purposes are different. Some classes may hold the data of the app, while others only define UI elements (View), while other classes control the communication between these two (Controller).

Python, one of the most versitile and friendly programming languages, presents itself as a great choice for the *Frontend* of the app, specially because of its Object Oriented workflow, it simplifies a lot of the complex data being thrown around the app. The *Backend* is defined in both Python and Julia. Specifically, Python stores the data, and julia, the creation of the linear system.

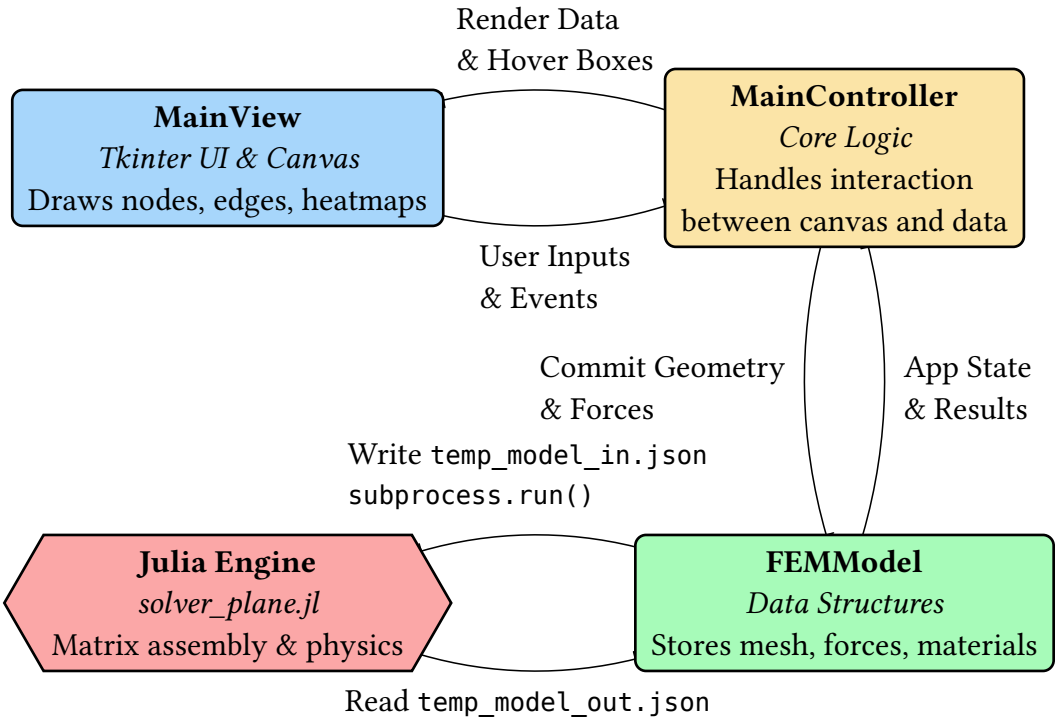


Figure 1: Flowchart of the app

3 CaST Workflow

3.1 Definition of Geometry

The first step of any FEM software is to define the geometry of the problem, not the mesh in itself, but the components that determine the format, material, and boundaries of the object.

In **CaST**, the user must firstly define **Nodes**, that store position along the space and will be connected via **Edges**. To define a node, one can either left click on the wanted position with the mouse (adjusting snap precision scrolling while holding shift) or type the coordinates manually (this, of course, ignores snap altogether). With the nodes placed, the user must connect 2 or 3 of them with the **Edge** tool. **CaST** supports linear, parabolic (isoparametric) and circular edges. Simply clicking in the order [start, end, middle] will define an edge. To define a whole, just change the boundary option above the canvas to a new hole.

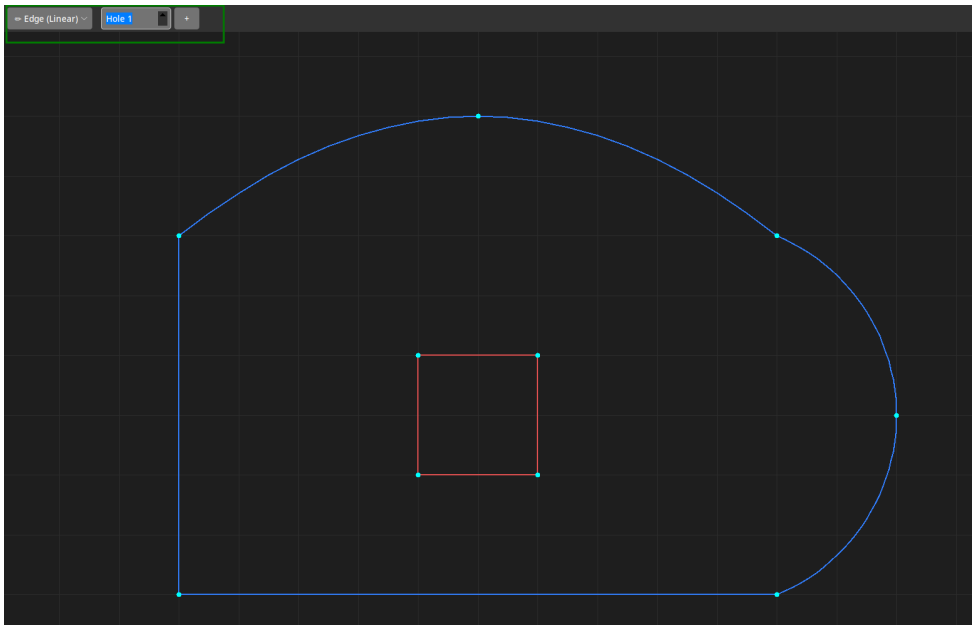


Figure 2: Definition of an object with linear, parabolic and circular edges, along with a linear hole. (Options menu, defined in green)

3.2 Adding of Supports and Forces

To define boundary conditions, user must click the respective node, if in node mode, or two nodes of a given edge, in edge mode. User may change displacement restrictions on the upper menu.

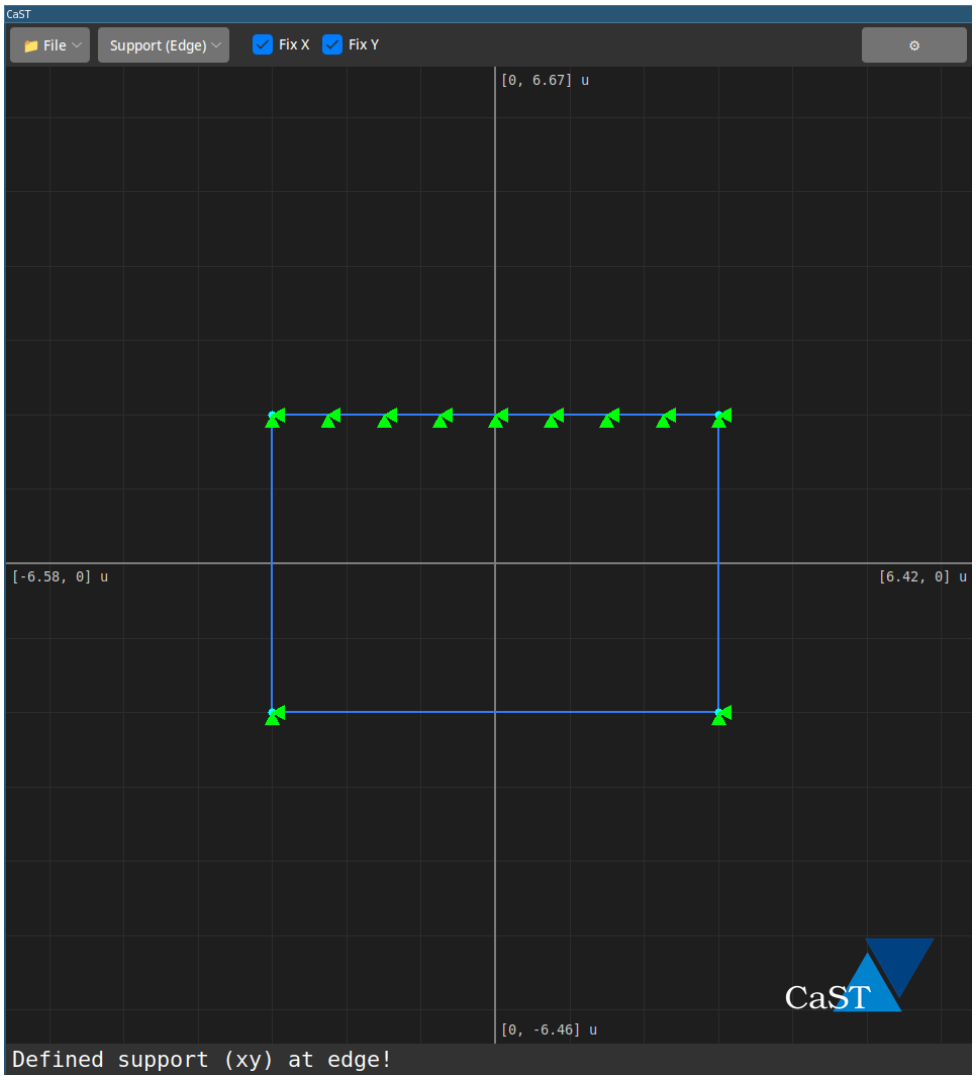


Figure 3: A rectangular object defined with supports at nodes (lower nodes) and edges (upper edge)

Similarly, for forces, they can be applied on nodes or edges. Edges can have “normal” (perpendicular to the element) and “global” (global coordinate system). Trapezoidal loads can be defined by differentiating start and end loads.

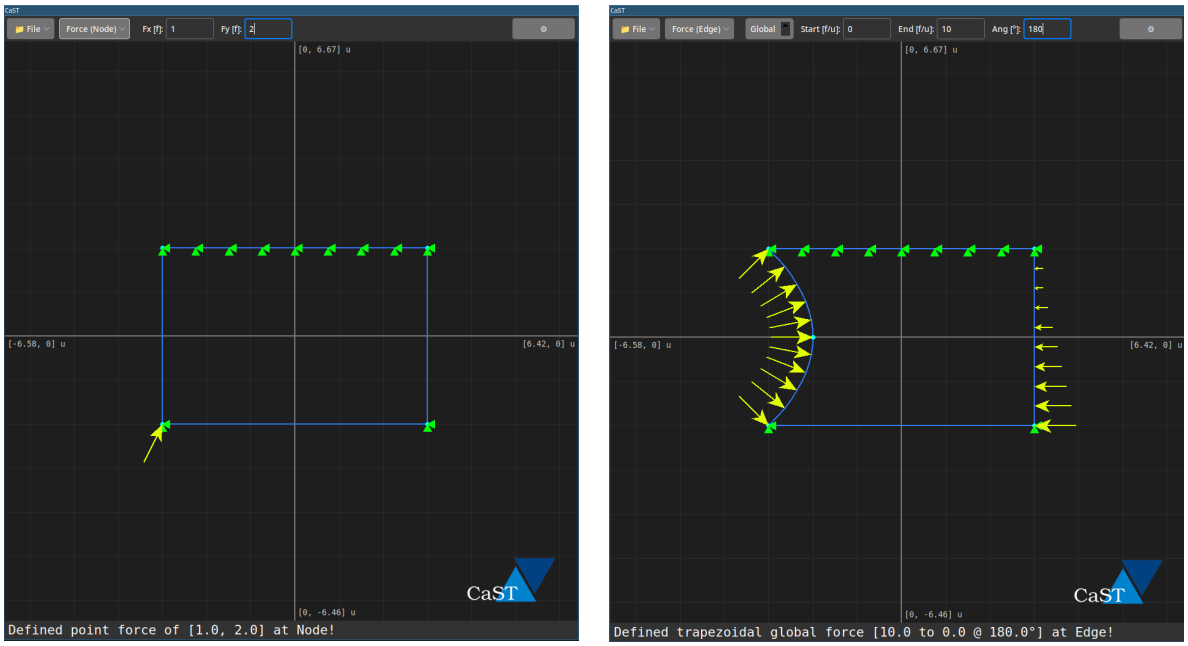


Figure 4: Definition of (a): Point force on node, (b): normal {left} and global trapezoidal load {right}

3.3 Material

The material consists of E (Young's module), ν (Poisson's ratio) and γ (Own Weight, on force per volume). On the material setting, user may also define the width of the object even though it's technically not a material property.

3.4 Meshing Engine

In Mesh mode, a simple scale defines the expected size of an element, to be updated on the mesh engine. Smaller elements tend to lag out the software, due to it redrawing every element on every frame.

Mesh mode also defines the analysis to be made, *Plane Stress* and *Plane Strain* (which changes specifically the D matrix, though more analysis are possible to be added).

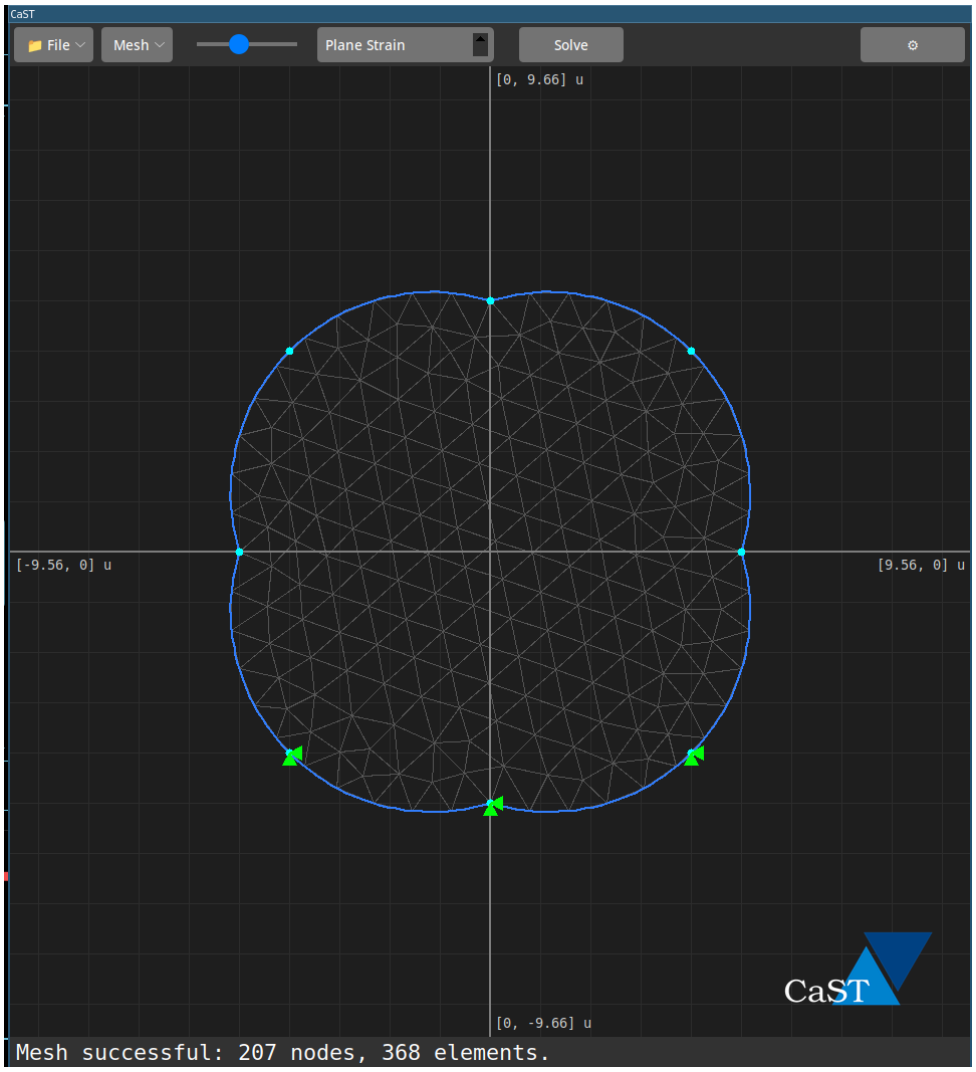


Figure 5: Mesh define for an element to be solved in Plane Strain

To solve complex geometries, the continuous physical domain must be discretized into a finite network of elements. While simple domains can be meshed manually, arbitrary geometries—especially those containing curves, parabolas, or internal holes—require specialized algorithmic generation. Tools like **Gmsh** (an open-source 3D finite element mesh generator) handle this discretization process. The meshing pipeline generally follows a strict topological hierarchy:

- **Points:** The absolute bounds of the model.
- **Curves:** Lines or arcs connecting the points to form wireframes.
- **Surfaces:** Closed boundaries formed by the curves, defining the actual solid material domain.

The below script (Listing 1) defines a pseudo-code that represents how the geometric CaST definitions are converted into useful FEM data, defining the mesh, and calculates the equivalent nodal forces and applies supports. **SolverNodes** is a dataclass that not only store its own coordinates, but support and force being applied.

```
MeshEngine(mesh_size)
  gmsh.initialize()

  # ---- DEFINE GEOMETRY ----
  for node in model.nodes:
    add node to gmsh
    define a tag for it

  for edge in model.edges:
    get its type (external or hole)
    if line:
      create straight line in gmsh between start and end node
    if parabola or circle:
      calculate intermediate points along the curve
      add these interpolated points as nodes in gmsh
      create a spline connecting start -> intermediate points -> end

  # ---- DEFINE BOUNDARIES & SURFACE ----
  for each boundary_group (external and holes):
    sort curves head-to-tail to form a closed, continuous loop
    create a CurveLoop in gmsh

  create a 2D PlaneSurface (using the external loop as the boundary, minus the hole
loops)

  # ---- MESH GENERATION ----
  generate 2D mesh!

  # ---- EXTRACT MESH DATA ----
  get all generated triangles and their associated node tags
  find only the "active" nodes (ignore unused points, like the center of a circle)
  create a new 0-based index for these active nodes (so the Julia solver gets a clean
array)

  # ---- MAP BOUNDARY CONDITIONS ----
  for each gmsh edge:
    find the actual mesh nodes generated along this specific edge
    apply edge supports to these nodes
    if edge has distributed load:
      split load across the edges mesh segments (trapezoidal integration)
      convert to X/Y directions and apply to the nodes

  for each base node:
    apply point supports and point loads directly to their matching gmsh node

  # ---- PACKAGE FOR SOLVER ----
  for each active node:
    create a SolverNode containing (x, y, support, fx, fy, moment)

  gmsh.finalize()

  return SolverNodes, Triangles
```

Listing 1: Representative script that converts geometric values into useful data for building the linear system.

Once the surface is defined, the meshing engine applies algorithms (such as Delaunay triangulation or advancing front methods) to pack the surface with non-overlapping triangles. A critical feature of advanced meshing is localized refinement . The engine

automatically generates smaller, denser triangles around areas of high geometric complexity (like sharp corners, holes, or applied point loads) to capture rapid stress concentrations accurately. Conversely, it leaves larger elements in uniform, unconstrained regions to reduce the total degrees of freedom, optimizing the computational speed of the linear solver.

To press the solve button on the top UI invokes the *Julia Engine* and sends the user to the analysis of results, which will be covered in Section 4.

3.5 CST Elements

The core geometric building block of the solver is the Constant Strain Triangle (CST), a 3-node, 6-degree-of-freedom element. The fundamental advantage of the CST lies in its linear displacement field. For any point inside the element, the displacements u_x and u_y are interpolated from the nodal displacements using linear shape functions:

$$u_x = N_1(x,y)u_{x_1} + N_2(x,y)u_{x_2} + N_3(x)u_{x_3}$$

$$u_y = N_1(x,y)u_{y_1} + N_2(x,y)u_{y_2} + N_3(x)u_{y_3}$$

Which implies that $u(x,y)$ is linear. Because strain is defined as the first spatial derivative of displacement (e.g., $\varepsilon_x = \frac{\partial u}{\partial x}$), taking the derivative of a linear function yields a constant:

$$\varepsilon = \text{cte}$$

This mathematical property guarantees that the strain—and consequently, the stress—does not vary across the element's internal domain. Computationally, this is a massive advantage. Defining the element stiffness matrix K_e requires integrating the material properties over the element's volume:

$$K_e = \int_V B^T D B dV$$

Because the strain-displacement matrix B is entirely constant for a CST, given the Jacobian J is too contant, it can be pulled outside of the integral. The calculation collapses from a complex numerical integration into a direct algebraic product involving the material's elasticity matrix D and the triangle's area A:

$$K_e = B^T D B A h$$

(where h is the element thickness). This eliminates the need for expensive numerical integration techniques, like Gaussian quadrature. As a result, the assembly of the global stiffness matrix in the backend becomes incredibly fast and memory-efficient, specially with the use of julia and its *SparseArrays* library.

Furthermore, the geometric simplicity of the CST easily translates to the application's physical parameters and interactive UI. Body forces, such as the specific weight of a material, can be perfectly distributed as three equal nodal loads calculated directly from the area.

4 Example Results

For the comparison of results of CaST and other FEM softwares, we will use the example set in the image below, from this subjects's textbook. The material has the following properties, for values in f, u:

$$E = 216 \cdot 10^9 \frac{f}{u^2}$$

$$\nu = 0.3$$

$$h = 0.005 \quad \text{u}$$

$$q = 50 \cdot 10^6 \cdot 5 \cdot 10^{-}3 = 250000 \frac{f}{u}$$

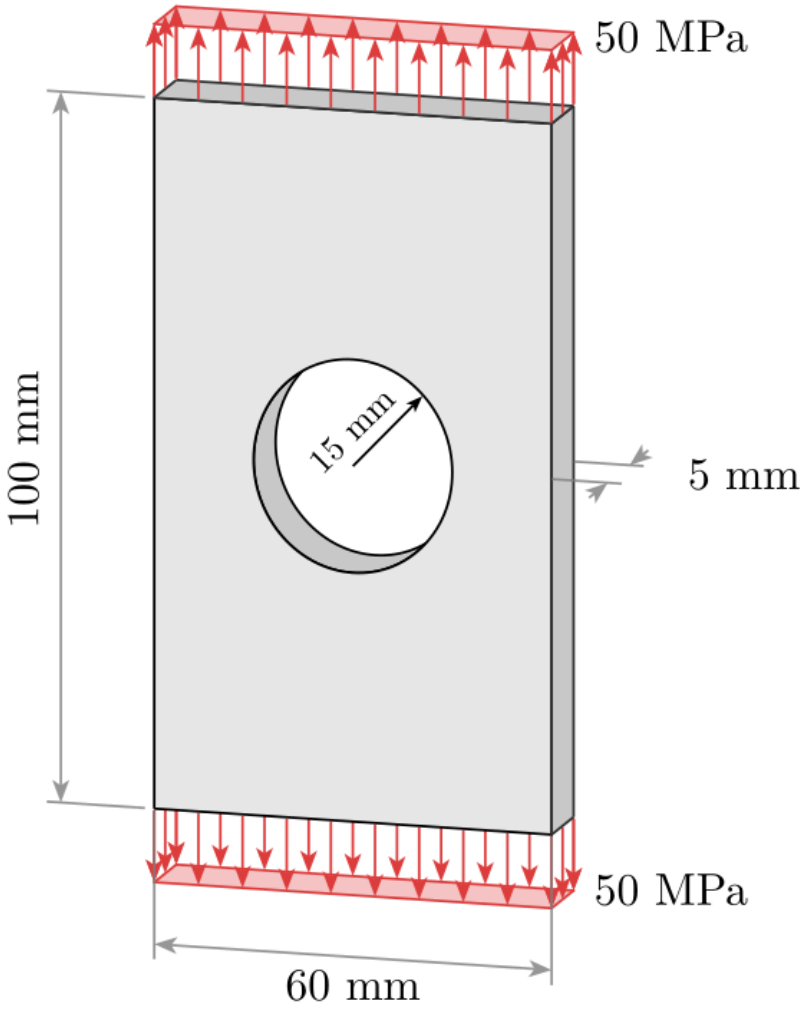


Figure 6: Symmetric plate to be solved

Applying symmetry, the plate in Figure 6 is drawn in CaST.

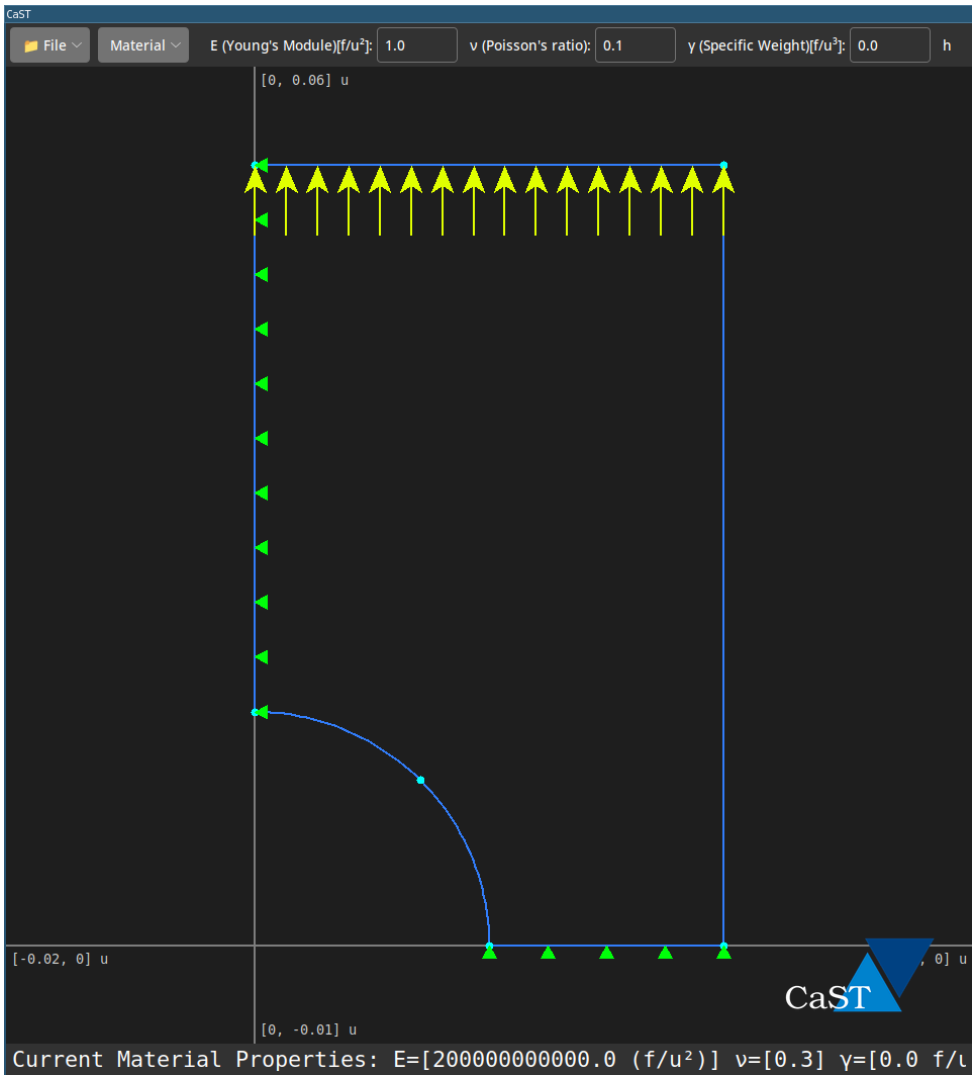


Figure 7: Example drawn at CaST

The element was discretized with 576 elements, and the Plane Stress analysis was chosen (given the place is thin). After pressing the solve button, the following result, for average displacements, appears. From testing, it was observed that:

- The computational time is more so because of the communication of the julia engine from python than the calculations itself.
- Efficiency on time varies a lot between which device was used.

Both of these observations will be observed in future versions of CaST.

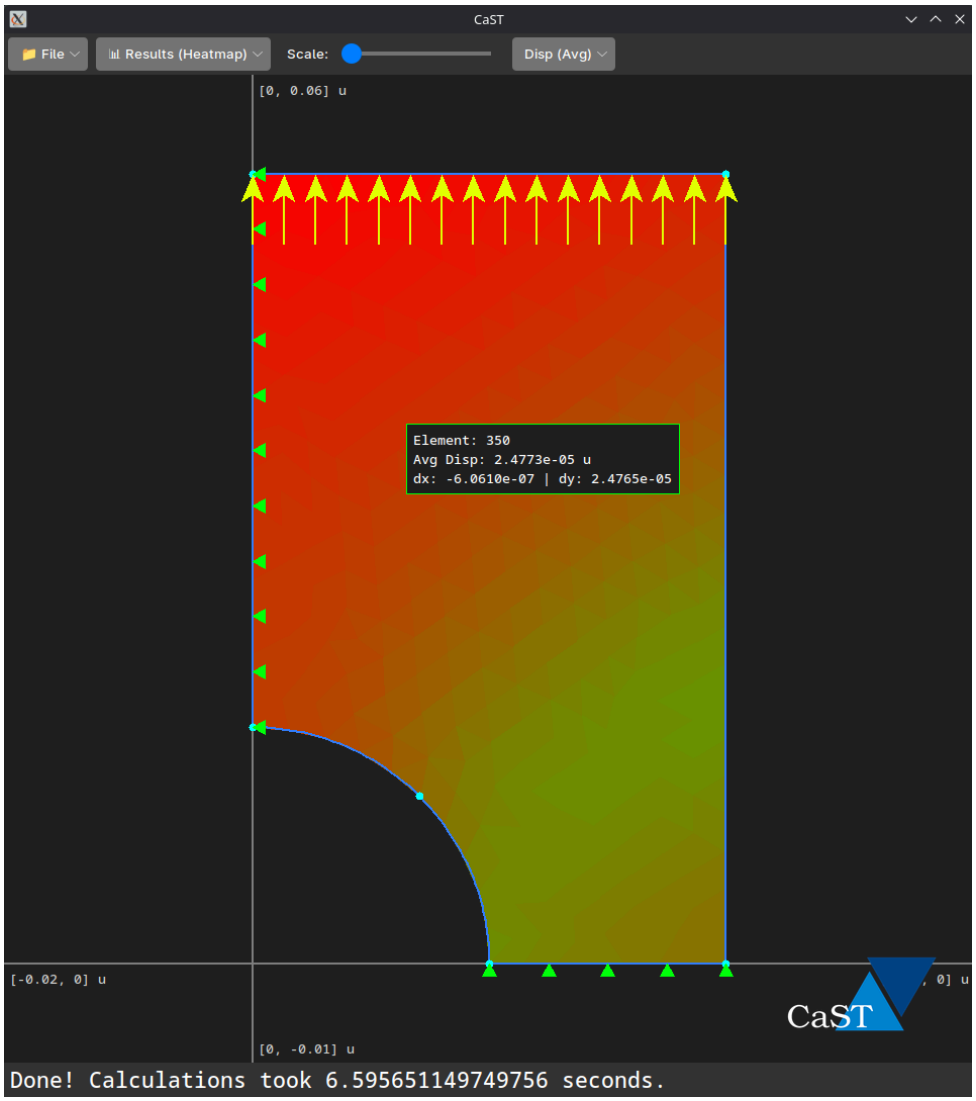


Figure 8: Results for average displacement

The figure shows a heatmap of average displacements, along with a scaled representation of displacements, to be changed with the slides above. Hovering the mouse shows the information on that current element.

Now, superficial comparison is made, between results on CaST and textbook.

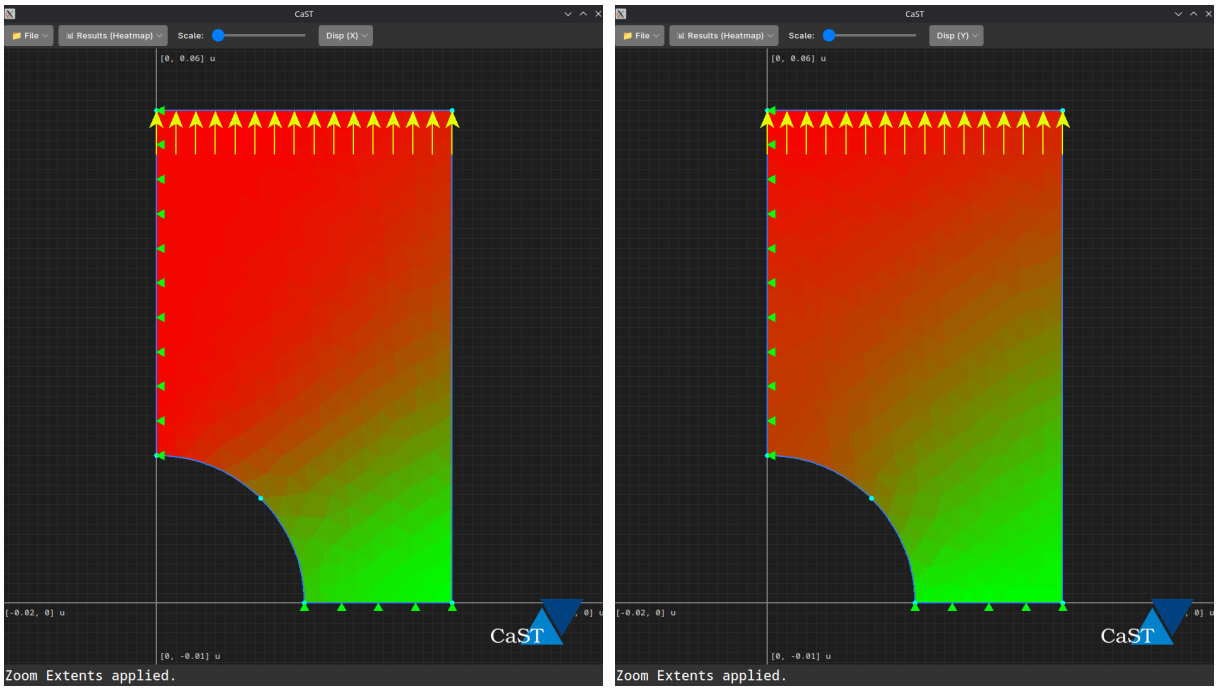
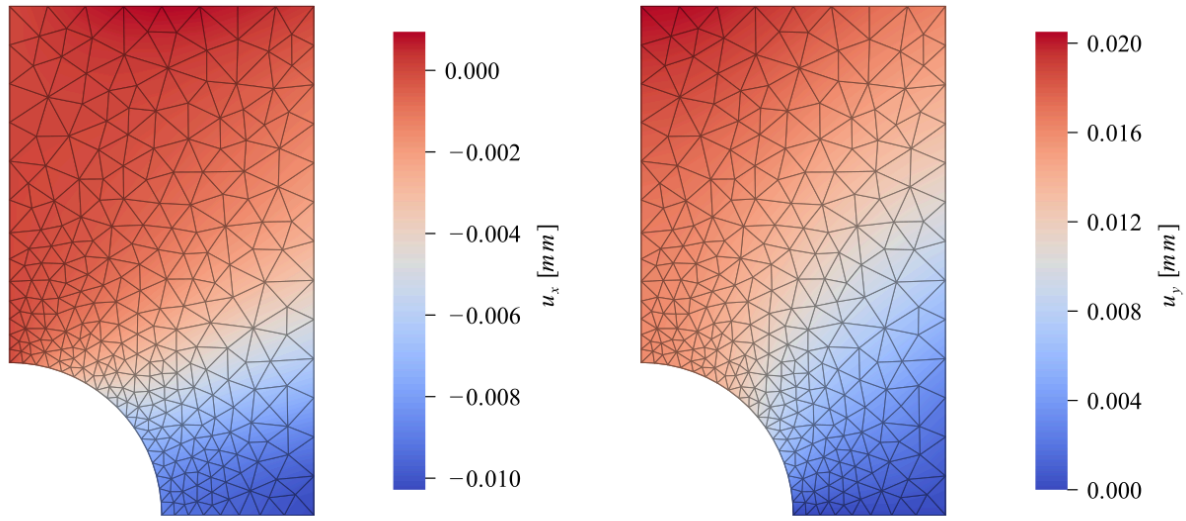


Figure 9: displacement heatmaps for x and y, respectively



u_x displacement field u_y displacement field

Figure 10: Solution as defined by the textbook, for displacements and x and y, respectively

As seen from Figure 9 and Figure 10, the heatmaps display very similar looking results. Numerically, though, there are differences.

For u_x, values in the graph vary from $\mathbf{0u}$ (at the boundary conditions) to about $\mathbf{-1.67 \cdot 10^{-5}u}$, Where u represents meters. So, it is clear CaST's solution is a tad exagerated (assuming the textbook as absolutely correct), and brings a lesser stiffness compared to the textbook's solution, which found the most exagerated displacement at about $1 \cdot 10^{-5}u$.

The same occurs for u_y. Values in the heatmap vary from $\mathbf{0u}$, at the boundary conditions, to about $\mathbf{3 \cdot 10^{-5}u}$, at the top left corner, where in the textbook are no more than $2 \cdot 10^{-5}u$.

Although, these results do not imply the software is incorrect; the visually similar gradient fields validate the underlying mathematical core of the engine, including the Jacobian transformations and the assembly of the global stiffness matrix.

In finite element theory, Constant Strain Triangles (CSTs) inherently suffer from artificial stiffness (often referred to as shear locking in bending-dominated scenarios). Therefore, if the discrepancy were solely due to the element formulation, CaST's mesh would typically underestimate the displacements, converging to the exact solution from below. Since CaST's model exhibits more flexible behavior (yielding larger displacements), the scalar shift is most likely attributed to force discretization methodologies and boundary meshing.

CaST computes nodal forces by interpolating the 1D line load across the boundary edges, using a segmented tributary method.If the textbook's reference software utilizes a different load integration technique—such as exact Gaussian quadrature evaluated over higher-order element boundaries—or applies its symmetry constraints over a different nodal density, the total effective applied force and boundary rigidity will marginally shift. Thus, the numerical difference highlights a variation in boundary interaction modeling rather than a failure in the solver's elasticity physics.

These results point of course to a conclusion that CaST is not (*yet*) a professionally designed software, and its use in professional enviroments is absolutely not recommended other than a superficial and quick visualization of the probable deformations of the element.

4.1 Stress results

Here presented are the results for stress distribution along the element.

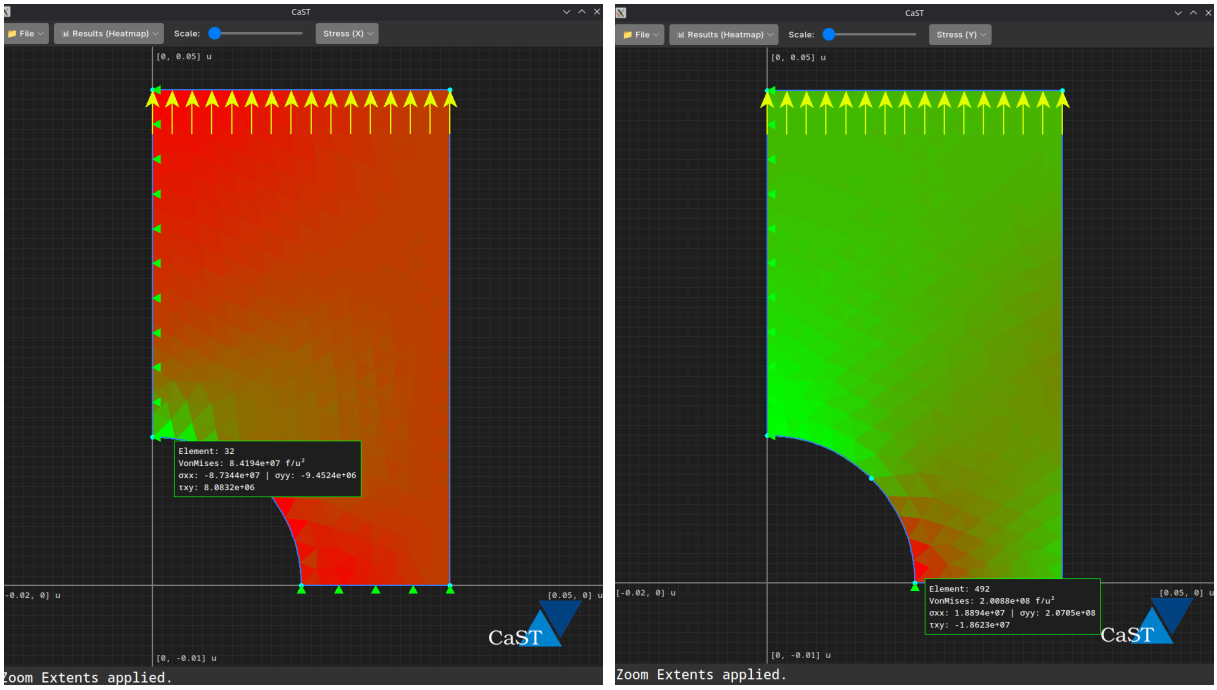


Figure 11: Stress heatmaps for x and y, respectively

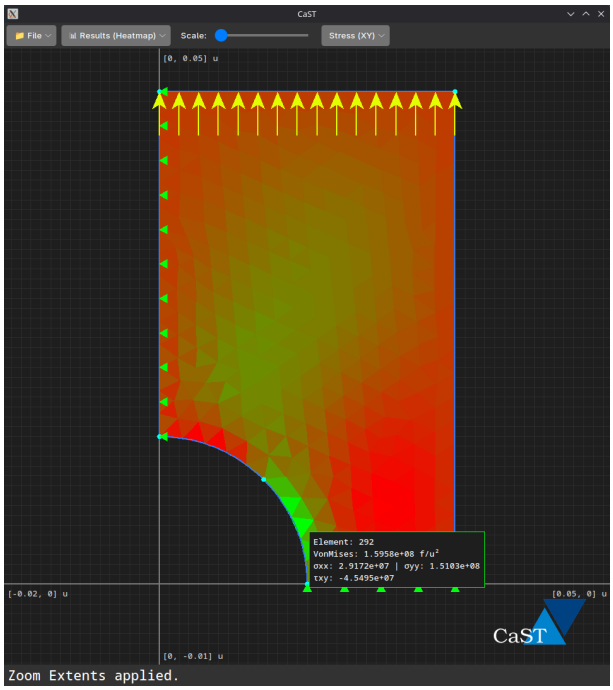


Figure 12: Shearing stress heatmap

The results seen from Figure 11 and Figure 12 are compatible with the expected, i.e. a tendency of “tearing” of the plate along the lower boundary, along with a higher compression of the element along the upper part of the circular hole.

An immediate improvement to the software would be a smoothing of the stress heatmap, improving the calculation of stress by considering not only the element itself, but its neighbors.

4.2 Node Analysis

CaST also allows the user to analyze nodes, which is specially useful to obtain information on reactions.

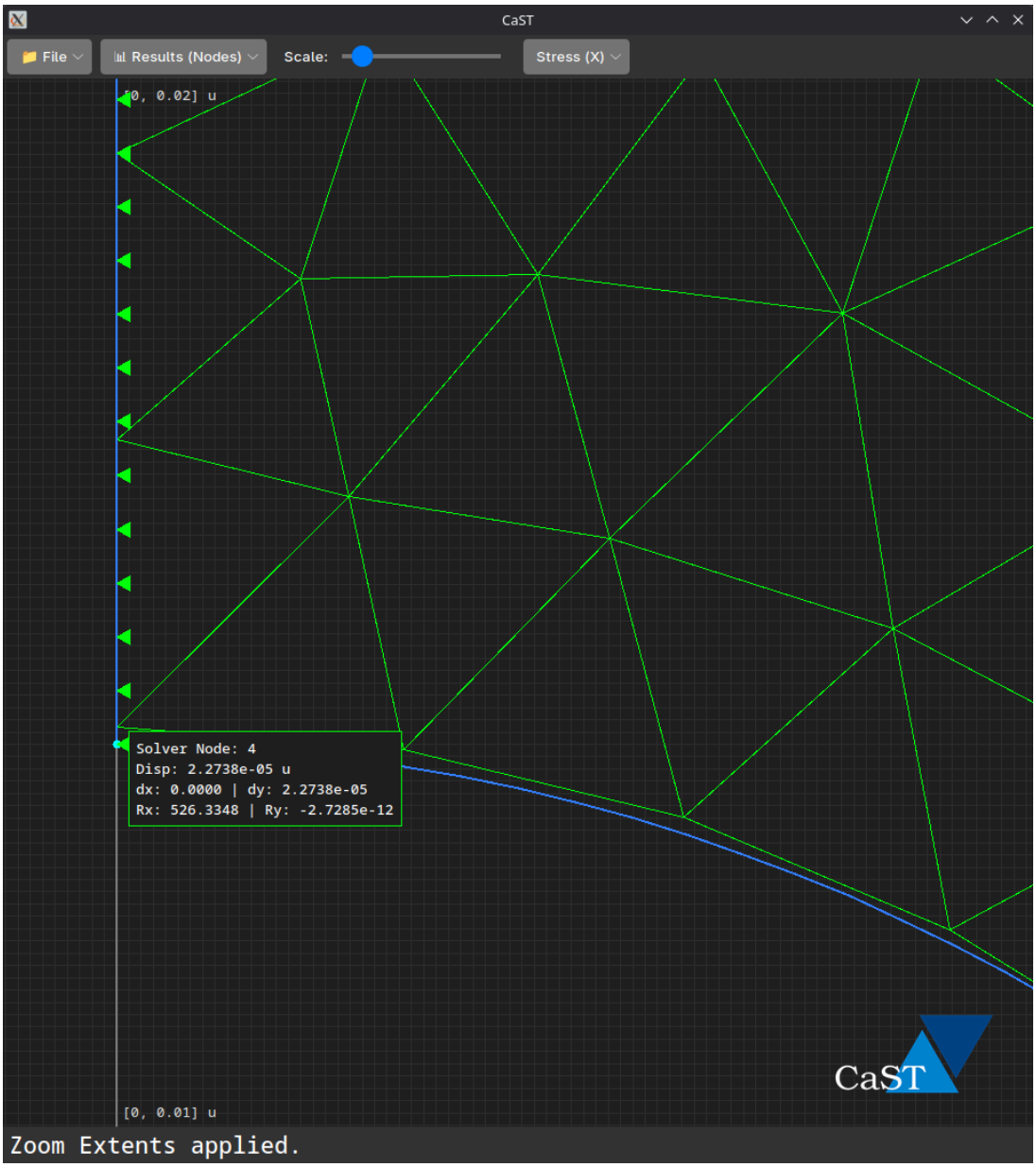


Figure 13: Shearing stress heatmap

From Figure 13, which zooms into the node from the upper section of the circular hole, we can obtain its id (Although not useful for much, other than debugging), average and axis displacements, and for support nodes, the reactions obtained. The node at the figure is compatible with expected results.

5 Conclusions

The development of CaST served as a comprehensive exercise in understanding and implementing the Finite Element Method from the ground up. The project successfully bridged theoretical solid mechanics with practical software engineering, demonstrating the complete computational pipeline.

While the current iteration of the software successfully captures the qualitative mechanical behavior of 2D elastic structures, several improvements could be implemented to enhance

its quantitative accuracy and overall capability. Conceivable upgrades for future iterations include:

- **Higher-Order Elements:** Implementing 6-node Linear Strain Triangles (LSTs) or quadrilateral elements to completely mitigate the artificial stiffness inherent to CSTs and accurately capture bending gradients.
- **Advanced Numerical Integration:** Upgrading the current tributary load distribution method to utilize exact Gaussian quadrature evaluated over the element boundaries, ensuring mathematically rigorous force discretization.
- **Better Mesh Refinement:** Integrating an automated feedback loop with the meshing engine to dynamically increase node density specifically around areas of high stress concentration or geometric complexity (such as holes and sharp corners).
- **Inter-Process Communication (IPC) Optimization:** Transitioning the data transfer between the Python frontend and the Julia solver from a standard json based file I/O to a faster, memory-based protocol—such as local sockets—to reduce latency.

Ultimately, CaST stands as a highly successful educational tool that validates the core formulations of linear elasticity while leaving a clear, modular pathway for advanced numerical expansions.

5.1 Repository

Code may be downloaded from the github repo below (careful, it's a mess):

https://github.com/Pepinopenguim/FEMapp,